

Лабораторная работа №7

Введение в работу с данными

Чемоданова Ангелина Александровна

1 ноября 2025

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

- Чемоданова Ангелина Александровна
- Студентка НФИбд-02-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132226443@pfur.ru
- <https://github.com/aachemodanova>



Цель работы

Основной целью работы является специализированных пакетов Julia для обработки данных.

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы.

В Julia для работы со структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO:

<pre>using Pkg</pre> ✓ 8.6s	Julia
<pre>using CSV</pre> ✓ 0.5s	Julia
<pre>using DataFrames</pre> ✓ 5.0s	Julia
<pre>using DelimitedFiles</pre> ✓ 0.1s	Julia

Рис. 1: Установка пакетов

Считывание данных

```
# Считывание данных и их запись в структуру:  
P = CSV.File("programminglanguages.csv") |> DataFrame
```

Julia

```
# Функция определения по названию языка программирования года его создания:  
function language_created_year(P, language::String)  
    loc = findfirst(P[:,2].==language)  
    return P[loc,1]  
end
```

Julia

language_created_year (generic function with 1 method)

```
language_created_year(P, "Python")
```

Julia

1991

Рис. 2: Считывание данных и запись в структуру

```
language_created_year(P, "Julia")
```

Julia

2012

Рис. 3: Пример

```
language_created_year(P,"julia")
```

Julia

MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)

The function `getindex` exists, but no method is defined for this combination of argument types.

Closest candidates are:

```
getindex(::DataFrame, !Matched::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
```

```
@ DataFrames C:\Users\angelina\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:542
```

```
getindex(::DataFrame, !Matched::typeof(!), ::Union{Signed, Unsigned})
```

```
@ DataFrames C:\Users\angelina\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:548
```

```
getindex(::DataFrame, !Matched::AbstractVector, ::Union{AbstractString, Signed, Symbol, Unsigned})
```

```
@ DataFrames C:\Users\angelina\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:530
```

...

...

```
@ Main c:\Users\angelina\Downloads\jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_W5sZmlsZQ==.jl:4
```

[2] top-level scope

```
@ c:\Users\angelina\Downloads\jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X11sZmlsZQ==.jl:1
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).

Рис. 4: Поиск “julia” со строчной буквы

```
# Функция определения по названию языка программирования  
# года его создания (без учёта регистра):  
  
function language_created_year_v2(P, language::String)  
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))  
    return P[loc,1]  
end
```

Julia

language_created_year_v2 (generic function with 1 method)

```
# Пример вызова функции и определение даты создания языка julia:  
language_created_year_v2(P, "julia")
```

Julia

2012

Рис. 5: Изменение исходной функции

```
# Построчное считывание данных с указанием разделителя:  
Tx = readlm("programminglanguages.csv", ',',')
```

Julia

```
74x2 Matrix{Any}:  
  "year"  "language"  
1951      "Regional Assembly Language"  
1952      "Autocode"  
1954      "IPL"  
1955      "FLOW-MATIC"  
1957      "FORTRAN"  
1957      "COMTRAN"  
1958      "LISP"  
1958      "ALGOL 58"  
1959      "FACT"  
  ⋮  
2007      "Clojure"  
2009      "Go"  
2010      "Rust"  
2011      "Dart"  
2011      "Kotlin"  
2011      "Red"  
2011      "Elixir"  
2012      "Julia"  
2014      "Swift"
```

Рис. 6: Построчное считывание данных

Запись данных в файл

```
# Запись данных в CSV-файл:  
CSV.write("programming_languages_data2.csv", P)
```

Julia

"programming_languages_data2.csv"

Рис. 7: Запись данных в файл

```
# Пример записи данных в текстовый файл с разделителем ',':  
writedlm("programming_languages_data.txt", Tx, ',')  
# Пример записи данных в текстовый файл с разделителем '~':  
writedlm("programming_languages_data2.txt", Tx, '~')
```

Julia

Рис. 8: Пример с указанием типа данных и разделителем данных

Запись данных в файл

```
# Построчное считывание данных с указанием разделителя:  
P_new_delim = readlm("programming_languages_data2.txt", '-')
```

Julia

```
74×2 Matrix{Any}:  
  "year"  "language"  
1951      "Regional Assembly Language"  
1952      "Autocode"  
1954      "IPL"  
1955      "FLOW-MATIC"  
1957      "FORTRAN"  
1957      "COMTRAN"  
1958      "LISP"  
1958      "ALGOL 58"  
1959      "FACT"  
  ⋮  
2007      "Clojure"  
2009      "Go"  
2010      "Rust"  
2011      "Dart"  
2011      "Kotlin"  
2011      "Red"  
2011      "Elixir"  
2012      "Julia"  
2014      "Swift"
```

Рис. 9: Проверка корректности считывания созданного текстового файла

```
# Инициализация словаря:  
dict = Dict{Integer,Vector{String}}()
```

Julia

```
Dict{Integer, Vector{String}}()
```

Рис. 10: Инициализация словаря

```
# Инициализация словаря:  
dict2 = Dict{}
```

Julia

```
Dict{Any, Any}()
```

Рис. 11: Инициализация пустого словаря

```
# Заполнение словаря данными:
for i = 1:size(P,1)
    year,lang = P[i,:]
    if year in keys(dict)
        dict[year] = push!(dict[year],lang)
    else
        dict[year] = [lang]
    end
end
```

Julia

Рис. 12: Заполнение словаря данными

```
# Пример определения в словаре языков программирования, созданных в 2003 году:
dict[2003]
```

Julia

```
2-element Vector{String}:
"Groovy"
"Scala"
```

Рис. 13: Пример работы словаря


```
# Задаём переменную со структурой DataFrame:
df = DataFrame(year = P[:,1], language = P[:,2])
```

73×2 DataFrame Julia
48 rows omitted

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT
10	1959	COBOL
11	1959	RPG
12	1962	APL
13	1962	Simula
⋮	⋮	⋮
62	2003	Scala
63	2005	F#
64	2006	PowerShell
65	2007	Clojure
66	2009	Go
67	2010	Rust
68	2011	Dart
69	2011	Kotlin
70	2011	Ruby

Рис. 14: Пример создания структуры DataFrame

```
# Вывод всех значения столбца year:  
df[:,year]
```

Julia

73-element Vector{Int64}:

1951
1952
1954
1955
1957
1957
1958
1958
1959
1959
:
2007
2009
2010
2011
2011
2011
2011
2012
2014

```
# Получение статистических сведений о фрейме:  
describe(df)
```

Julia

2×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	year	1982.99	1951	1986.0	2014	0	Int64
2	language		ALGOL 58		dBase III	0	String31

◀

▶

Рис. 15: Пример создания структуры DataFrame

```
# Подгружаем пакет RDatasets:  
using RDatasets
```

Julia

```
# Задаём структуру данных в виде набора данных:  
iris = dataset("datasets", "iris")
```

Julia

150×5 DataFrame

125 rows omitted

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
⋮	⋮	⋮	⋮	⋮	⋮
139	6.0	3.0	4.8	1.8	virginica
140	6.9	3.1	5.4	2.1	virginica
141	6.7	3.1	5.6	2.4	virginica
142	6.9	3.1	5.1	2.3	virginica
143	5.8	2.7	5.1	1.9	virginica

Рис. 16: Работа с пакетом RDatasets

```
# Определения типа переменной:  
typeof(iris)
```

Julia

DataFrame

```
describe(iris)
```

✓ 6.1s

Julia

5×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Float64	Real	Float64	Real	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species	2.0	1	2.0	3	0	Int64

Рис. 17: Получение основных статических сведений о каждом столбце в наборе данных

Работа с переменными отсутствующего типа (Missing Values)

```
# Отсутствующий тип:  
a = missing  
typeof(a)  
✓ 0.7s  
Missing  
  
# Пример операции с переменной отсутствующего типа:  
a + 1  
✓ 0.3s  
missing
```

Рис. 18: Использование “отсутствующего” типа. Операция сложения числа и переменной с отсутствующим типом

Работа с переменными отсутствующего типа (Missing Values)

```
# Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]
✓ 1.0s Julia
```

```
4-element Vector{Union{Missing, Int64}}:
 missing
    47
    22
   105
```

```
# Определение типа переменной:
typeof(calories)
✓ 0.1s Julia
```

```
Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рис. 19: Пример работы с данными, среди которых есть данные с отсутствующим типом

Работа с переменными отсутствующего типа (Missing Values)

<pre># Подключаем пакет Statistics: using Statistics # Определение среднего значения: mean(calories)</pre>	
✓ 0.2s	Julia
missing	
<pre># Определение среднего значения без значений с отсутствующим типом: mean(skipmissing(calories))</pre>	
	Julia
58.0	

Рис. 20: Игнорирование отсутствующего типа

Работа с переменными отсутствующего типа (Missing Values)

```
# Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]
# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)
# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods, price=prices)
# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories, dataframe_prices, on=:item)
```

Julia

4×3 DataFrame

Row	item	calories	price
	String	Int64?	Float64
1	apple	missing	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

Рис. 21: Формирование таблиц данных и их объединение в один фрейм


```
# Подключаем пакет FileIO:  
using FileIO  
using ImageIO
```

Julia

```
# Загрузка изображения:  
X1 = load("julialogo.png")
```

Julia

Рис. 22: Подключение пакетов. Загрузка изображения

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Загрузка данных:  
houses = CSV.File("houses.csv") |> DataFrame
```

Julia

985×12 DataFrame

960 rows omitted

Row	street	city	zip	state	beds	baths	sq_ft	type	sale_date	price	latitude	longitude
	String	String15	Int64	String3	Int64	Int64	Int64	String15	String31	Int64	Float64	Float64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	59222	38.6319	-121.435
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	68212	38.4789	-121.431
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	68880	38.6183	-121.444
4	2805 JANETTE WAY	SACRAMENTO	95815	CA	2	1	852	Residential	Wed May 21 00:00:00 EDT 2008	69307	38.6168	-121.439
5	6001 MCMAHON DR	SACRAMENTO	95824	CA	2	1	797	Residential	Wed May 21 00:00:00 EDT 2008	81900	38.5195	-121.436
6	5828 PEPPERMILL CT	SACRAMENTO	95841	CA	3	1	1122	Condo	Wed May 21 00:00:00 EDT 2008	89921	38.6626	-121.328
7	6048 OGDEN NASH WAY	SACRAMENTO	95842	CA	3	2	1104	Residential	Wed May 21 00:00:00 EDT 2008	90895	38.6817	-121.352
8	2561 19TH AVE	SACRAMENTO	95820	CA	3	1	1177	Residential	Wed May 21 00:00:00 EDT 2008	91002	38.5351	-121.481
9	11150 TRINITY RIVER DR Unit 114	RANCHO CORDOVA	95670	CA	2	2	941	Condo	Wed May 21 00:00:00 EDT 2008	94905	38.6212	-121.271
10	7325 10TH ST	RIO LINDA	95673	CA	3	2	1146	Residential	Wed May 21 00:00:00 EDT 2008	98937	38.7009	-121.443
11	645 MORRISON AVE	SACRAMENTO	95838	CA	3	2	909	Residential	Wed May 21 00:00:00 EDT 2008	100309	38.6377	-121.452
12	4085 FAWN CIR	SACRAMENTO	95823	CA	3	2	1289	Residential	Wed May 21 00:00:00 EDT 2008	106250	38.4707	-121.459
13	2930 LA ROSA RD	SACRAMENTO	95815	CA	1	1	871	Residential	Wed May 21 00:00:00 EDT 2008	106852	38.6187	-121.436
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
974	2181 WINTERHAVEN CIR	CAMERON PARK	95682	CA	3	2	0	Residential	Thu May 15 00:00:00 EDT 2008	224500	38.6976	-120.996
975	7540 HICKORY AVE	ORANGEVALE	95662	CA	3	1	1456	Residential	Thu May 15 00:00:00 EDT 2008	225000	38.7031	-121.235
976	5024 CHAMBERLIN CIR	ELK GROVE	95757	CA	3	2	1450	Residential	Thu May 15 00:00:00 EDT 2008	228000	38.3898	-121.446
977	2400 INVERNESS DR	LINCOLN	95648	CA	3	2	1358	Residential	Thu May 15 00:00:00 EDT 2008	229027	38.8978	-121.325
978	5 BISHOPGATE CT	SACRAMENTO	95823	CA	4	2	1329	Residential	Thu May 15 00:00:00 EDT 2008	229500	38.4679	-121.445
979	5601 REXLEIGH DR	SACRAMENTO	95823	CA	4	2	1715	Residential	Thu May 15 00:00:00 EDT 2008	230000	38.4453	-121.442
980	1909 YARNELL WAY	ELK GROVE	95758	CA	3	2	1262	Residential	Thu May 15 00:00:00 EDT 2008	230000	38.4174	-121.484
981	9169 GARLINGTON CT	SACRAMENTO	95829	CA	4	3	2280	Residential	Thu May 15 00:00:00 EDT 2008	232425	38.4577	-121.36
982	6932 RUSKUT WAY	SACRAMENTO	95823	CA	3	2	1477	Residential	Thu May 15 00:00:00 EDT 2008	234000	38.4999	-121.459

Рис. 23: Загрузка данных

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Построение графика:  
using Plots  
plot(size=(500,500),leg=false)  
x = houses[:,sq_ft]  
y = houses[:,price]  
scatter(x,y,markersize=3)
```

Julia

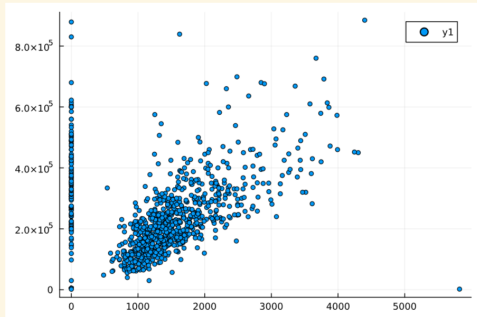


Рис. 24: Построение графика цен на недвижимость в зависимости от площади

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Фильтрация данных по заданному условию:  
filter_houses = houses[houses[:,sq_ft].>0,:]   
# Построение графика:  
x = filter_houses[:,sq_ft]  
y = filter_houses[:,price]  
scatter(x,y)
```

Julia

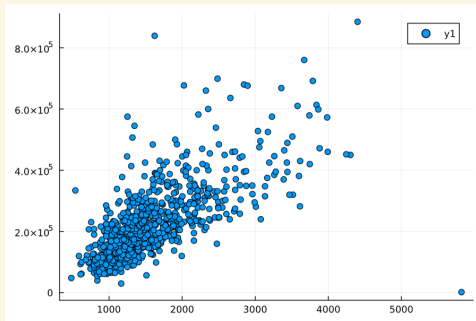


Рис. 25: Построение графика без “артефактов”

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
using Statistics
# Определение средней цены для каждого типа домов
mean_prices = combine(groupby(filter_houses, :type), :price => mean => :mean_price)
println("Средние цены для каждого типа домов:")
println(mean_prices)
```

Julia

Средние цены для каждого типа домов:

4x2 DataFrame

Row	type	mean_price
	String15	Float64
1	Residential	2.39186e5
2	Condo	1.50082e5
3	Multi-Family	2.24535e5
4	Unkown	275000.0

Рис. 26: Среднее значение

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Добавление данных :latitude и :longitude
X = filter_houses[:, [:latitude, :longitude]]
# Преобразование DataFrame в матрицу
X = Matrix{Float64}(X)'
k = length(unique(filter_houses[:, :zip]))
# Определение кластеров k-среднего
C = kmeans(X, k)
# Формирование DataFrame с результатами кластеризации
df = DataFrame(
    cluster = C.assignments,
    city = filter_houses[:, :city],
    latitude = filter_houses[:, :latitude],
    longitude = filter_houses[:, :longitude],
    zip = filter_houses[:, :zip]
)
# Построение графика кластеров
clusters_figure = plot(legend=false)
for i in 1:k
    clustered_houses = df[df[:, :cluster] .== i, :]
    xvals = clustered_houses[:, :latitude]
    yvals = clustered_houses[:, :longitude]
    scatter!(clusters_figure, xvals, yvals, markersize=4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)
```

Julia

Рис. 27: Построение графика с кластерами разных цветов

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

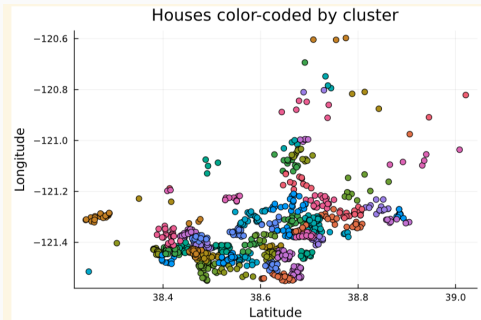


Рис. 28: Построение графика с кластерами разных цветов

Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
unique_zips = unique(filter_houses[:,zip])
zip_figure = plot(legend = false)
for uzip in unique_zips
    subs = filter_houses[filter_houses[:,zip].==uzip,:]
    x = subs[:,latitude]
    y = subs[:,longitude]
    scatter!(zip_figure,x,y)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by zip code")
display(zip_figure)
```

Julia

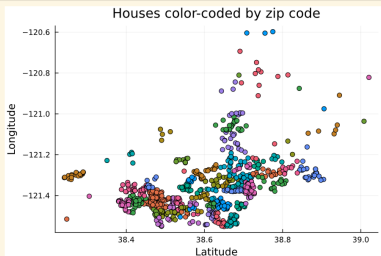


Рис. 29: Построение графика с кластерами разных цветов по почтовому индексу

Кластеризация данных. Метод k ближайших соседей

```
knearest = 10
id = 70
point = X[:,id]

# Поиск ближайших соседей:
kdtree = KDTree(X)
idxs, dists = knn(kdtree, point, knearest, true)

# Все объекты недвижимости:
x = filter_houses[:, :latitude];
y = filter_houses[:, :longitude];
scatter(x, y)
# Соседи:
x = filter_houses[idxs, :latitude];
y = filter_houses[idxs, :longitude];
scatter!(x, y)
```

Julia

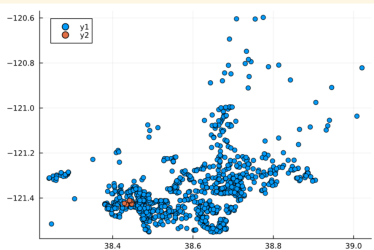


Рис. 30: Отображение на графике соседей выбранного объекта недвижимости

```
# Фильтрация по районам соседних домов:  
cities = filter_houses[idxs,:city]
```

Julia

```
10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:
```

```
"SACRAMENTO"  
"ELK GROVE"  
"SACRAMENTO"  
"SACRAMENTO"  
"SACRAMENTO"  
"SACRAMENTO"  
"ELK GROVE"  
"ELK GROVE"  
"ELK GROVE"  
"ELK GROVE"
```

Рис. 31: Определение районов соседних домов

Обработка данных. Метод главных компонент

```
# Фрейм с указанием площади и цены недвижимости:  
F = filter_houses[l, [:sq_ft,:price]]  
y = filter_houses[l, :price]  
F = Matrix(F)  
y = y'  
  
# Приведение типов данных к распределение для PCA:  
M = fit(PCA, F)  
  
# Выделение значений главных компонент в отдельную переменную:  
Xr = reconstruct(M, y)  
  
# Построение графика с выделением главных компонент:  
scatter(F[1,:],F[2,:])  
scatter!(Xr[1,:],Xr[2,:])
```

Julia

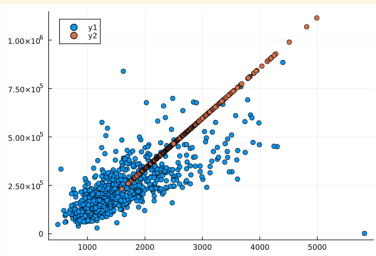


Рис. 32: Попытка уменьшения размера данных о цене и площади из набора данных домов

```
xvals = repeat(1:0.5:10,inner=2)
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1
scatter(xvals,yvals,color=:black,leg=false)
```

Julia

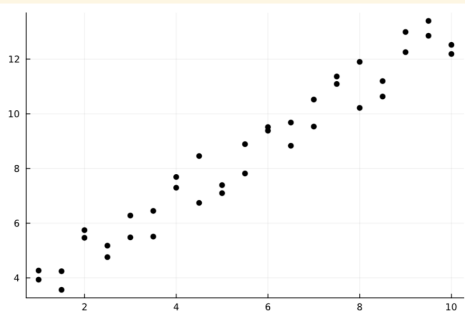


Рис. 33: Исходные данные

Обработка данных. Линейная регрессия

```
function find_best_fit(xvals,yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals,yvals)
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
end
```

✓ 0.3s

Julia

find_best_fit (generic function with 1 method)

```
a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot!(xvals,ynew)
```

Julia

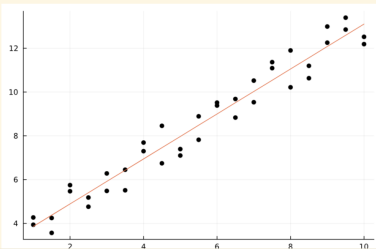


Рис. 34: Применение функции для построения графика

```
xvals = 1:100000;  
xvals = repeat(xvals,inner=3);  
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1;  
@show size(xvals)  
@show size(yvals)
```

Julia

```
size(xvals) = (300000,  
size(yvals) = (300000,  
  
(300000,)
```

```
@time a,b = find_best_fit(xvals,yvals)
```

Julia

```
0.089711 seconds (30.39 k allocations: 1.548 MiB, 97.69% compilation time)  
  
(1.0000000055752662, 3.0008961140774773)
```

```
using PyCall  
using Conda
```

Julia

Рис. 35: Генерация данных. Определение времени соответствия

Обработка данных. Линейная регрессия

```
py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""
xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = py"find_best_fit_python"(xpy,ypy)
```

Julia

0.575768 seconds (99.16 k allocations: 5.005 MiB, 41.58% compilation time)

(1.000000005575266, 3.000896114084753)

```
using BenchmarkTools
@btime a,b = py"find_best_fit_python"(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)
```

Julia

9.111 ms (32 allocations: 912 bytes)

1.116 ms (1 allocation: 32 bytes)

(1.0000000055752662, 3.0008961140774773)

Рис. 36: Сравнение

Самостоятельное выполнение

1.1) Кластеризация

```
using RDatasets, Clustering, Plots
```

✓ 21.3s

Julia

```
iris = dataset("datasets", "iris")
iris_spec = Dict{"setosa"=>1, "versicolor"=>2, "virginica"=>3}
iris[1, :5] = [iris_spec[item] for item in iris[1, :5]]

X = Matrix(iris)'
k = length(unique(iris[1, :5]))
C = kmeans(X, k)
df = DataFrame(cluster = C.assignments, SepalLength = iris[1, :SepalLength],
               SepalWidth = iris[1, :SepalWidth], PetalLength = iris[1, :PetalLength],
               PetalWidth = iris[1, :PetalWidth], Species = iris[1, :Species])

clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[1, :cluster].== i,:]
    xvals = clustered_houses[1, :SepalWidth]
    yvals = clustered_houses[1, :SepalLength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

✓ 0.3s

Julia

Рис. 37: Кластеризация

Самостоятельное выполнение

```
clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[:, :cluster].== i, :]
    xvals = clustered_houses[:, :SepalWidth]
    yvals = clustered_houses[:, :SepalLength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

✓ 0.3s

Julia

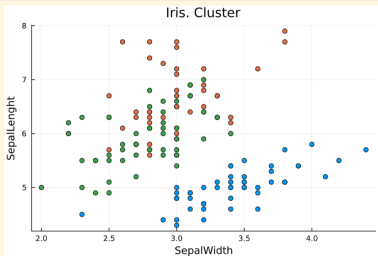


Рис. 38: Кластеризация

```
clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[:, :cluster].== i,:]
    xvals = clustered_houses[:, :PetalWidth]
    yvals = clustered_houses[:, :PetalLength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("SepalWidth")
ylabel!("SepalLength")
title!("Iris. Cluster")
display(clusters_figure)
```

✓ 0.1s

Julia

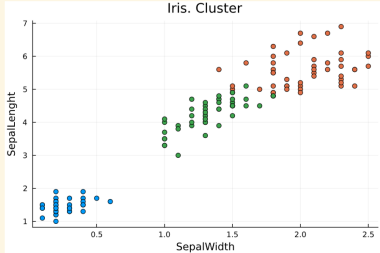


Рис. 39: Кластеризация

1.2) Регрессия (метод наименьших квадратов в случае линейной регрессии)

Часть 1.

```
x = randn(1000, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(1000)
scatter(X, y, vals = (1, 2, 3))
```

✓ 0.5s

Julia

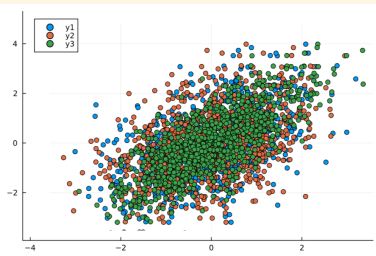


Рис. 40: Регрессия

<pre>X2 = hcat(ones(1000), X) beta = (X2' * X2) \ (X2' * y)</pre>	Julia
✓ 0.0s	
4-element Vector{Float64}: 0.0029871617603585755 0.6407236341578142 0.5666445893979948 0.9878329997096461	
<pre>using MultivariateStats beta_mvs = llsq(X, y)</pre>	Julia
✓ 0.0s	
4-element Vector{Float64}: 0.6407236341578141 0.5666445893979949 0.9878329997096466 0.00298716176035858	
<pre>using GLM df = DataFrame(hcat(X, y), :auto) rename!(df, ["x1", "x2", "x3", "y"]) model = lm(@Formula(y ~ x1 + x2 + x3), df) coef(model)</pre>	Julia
✓ 12.1s	
4-element Vector{Float64}: 0.0029871617603585755 0.6407236341578141 0.5666445893979949 0.9878329997096459	

Рис. 41: Регрессия

Часть 2.

```
X = rand(100)
y = 2X + 0.1 * randn(100)
a, b = find_best_fit(X, y)
scatter(X, y, c="pink", title = "График регрессии", xlabel='x', ylabel='y', leg = false)
Plots.abline!(a, b, lw = 2)
```

✓ 0.2s

Julia

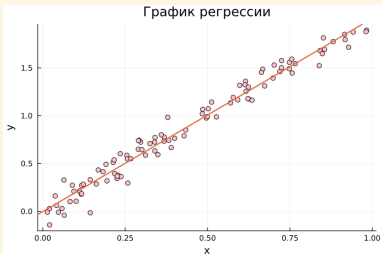


Рис. 42: Регрессия

1.3) Модель ценообразования биномиальных опционов

а.

```
S = 100.0      # начальная цена акции
T = 1.0        # длина биномиального дерева в годах;
n = 1000       # количество периодов
sigma = 0.3    # волатильность
r = 0.08       # годовая процентная ставка
h = T / n      # длина одного периода

u = exp(r * h + sigma * sqrt(h))
d = exp(r * h - sigma * sqrt(h))
p = (exp(r*h) - d) / (u - d)

function create_path(S, r, sigma, T, n)
    path = []
    S_ = S
    push!(path, S)
    for i in 1:n
        if rand() < p
            S_ *= u
        else
            S_ *= d
        end
        push!(path, S_)
    end
    return path
end

path = create_path(S, r, sigma, T, n)
plot(path, label="Stock Price Path", xlabel="Time Step", ylabel="Price", title="Stock Price Trajectory")
```

✓ 0.2s

Julia

Рис. 43: Модель ценообразования биномиальных опционов

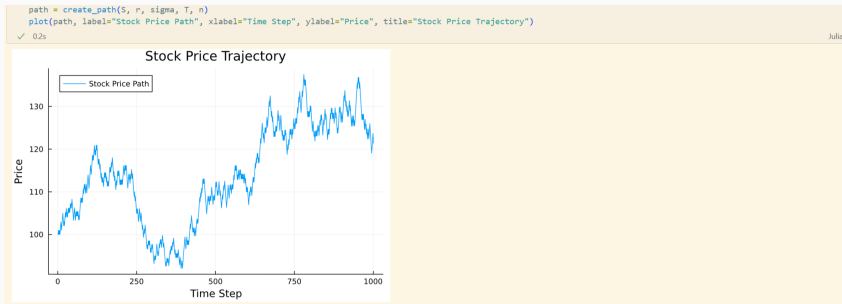


Рис. 44: Модель ценообразования биномиальных опционов

b.

```
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect{Float64}(0:n-1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r*h) - d) / (u - d)
    for i=0:n-1
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
end
p
```

✓ 0.7s

Julia

Рис. 45: Модель ценообразования биномиальных опционов

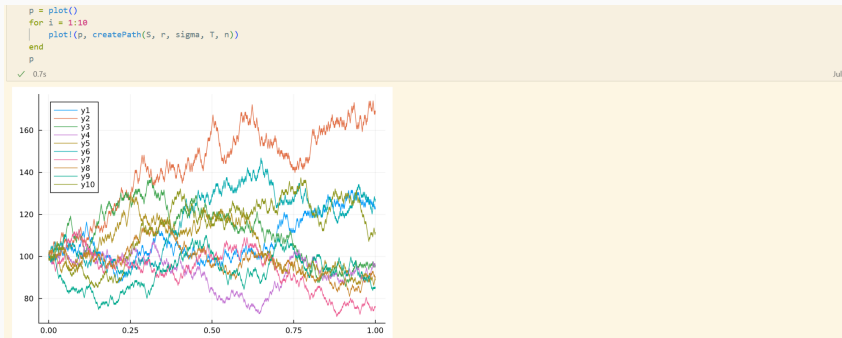


Рис. 46: Модель ценообразования биномиальных опционов

C.

```
function createPath_(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect{Float64}(0:T/n:1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r*h) - d) / (u - d)
    Threads.@threads for i=0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath_(S, r, sigma, T, n))
end
p
```

✓ 1.1s

Julia

Рис. 47: Модель ценообразования биномиальных опционов

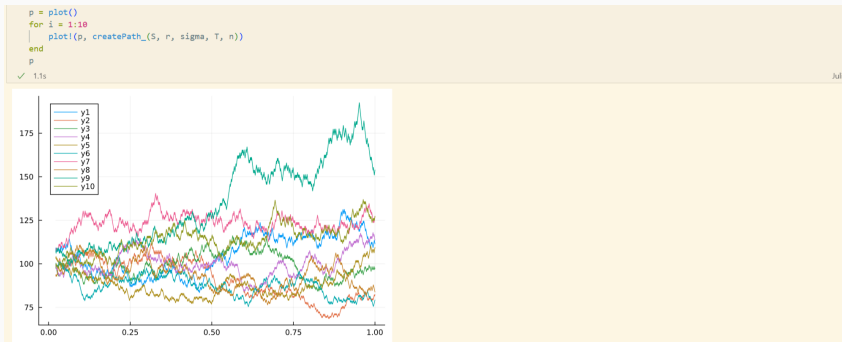


Рис. 48: Модель ценообразования биномиальных опционов

В ходе выполнения лабораторной работы были изучены специализированные пакеты Julia для обработки данных.